Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

The concepts from lectures and drafter practicums will help complete this project. I expect to use conditionals, loops, lists and classes when programming this project. I hope to improve my debugging skills and ability to read code in this project.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I expect the easiest part of the project to be coming up with an idea and fitting into the code requirements. There are not that many code requirements and I already have an idea for the project. I think writing the code and checking it will be the hardest. I think this because there will be a lot of code to check and the LLM can make mistakes.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

The task that will take the most time is coding the first part of my website. The part where the people first see and has all the selections on it. All the different Pages and States should be implemented first. Along with my ideas for each part of the website.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Other school work or exams might interfere with my work. I will minimize them by staying on top of all my work and working ahead in my other class. That way I have more time to work on my comp sci project. I will probably need an LLM, a TA, or even the professor to help fix the code. I have no remaining questions about the assignment requirements.

1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

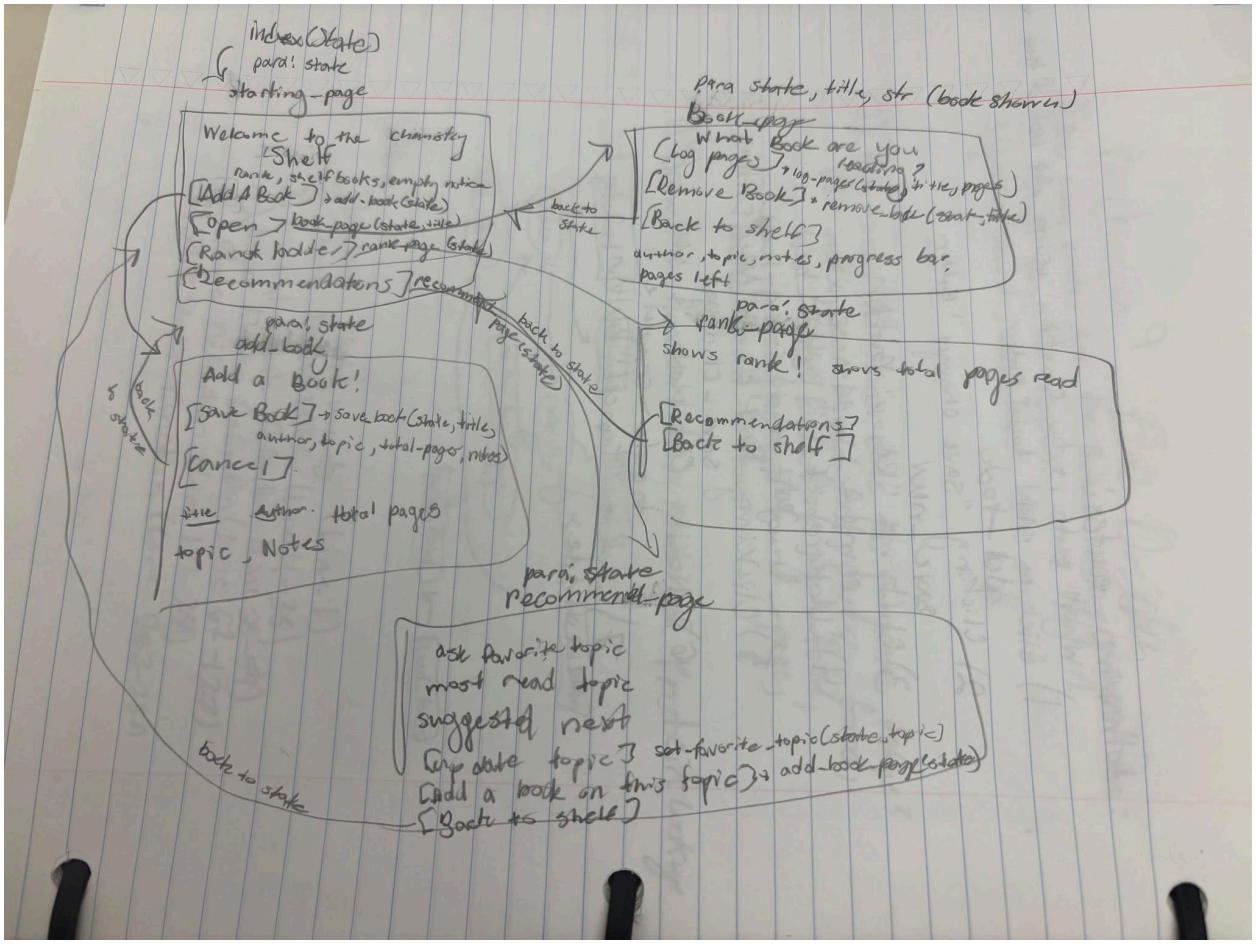
In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.

States:

```
class State:
    books: list[Books]
    total_pages_read: int
    rank: str
    favorite_topic: str
    message: str
```

```
class Book:
    title: str
    author: str
    topic: str
    total_pages: int
    pages_read: int
    notes: str
```


index(state)
para: state
starting_page
Welcome to the chemistry Shelf
[Add A Book] → add_book(state)
[Open] → book_page(state, title)
[Ranck ladder] → rank_page(state)
[Recommendations] → recommended_page(state)
back to state
para: state, title, str (book shown)
book_page
What book are you reading?
[Log pages] → log_pages(state, title, pages)
[Remove Book] → remove_book(state, title)
[Back to shelf]
author, topic, notes, progress bar, pages left
back to state
para: state
add_book
Add a Book!
[Save Book] → save_book(state, title, author, topic, total_pages, notes)
[Cancel]
title, author, total pages
topic, Notes
back to state
para: state
rank_page
shows rank! shows total pages read
[Recommendations]
[Back to shelf]
para: state
recommended_page
ask favorite topic
most read topic
suggested next
[Top state topic] → set_favorite_topic(state, topic)
[Add a book on this topic] → add_book_page(state)
[Back to shelf]
back to state

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	My site helps organize and for someone who wants to look at boring chemistry textbooks. The website will hold, take new books, let you read basic information about them, and give you a score based on how much you've read in a creative chemistry way by adding new topics to read and give recommendations for topics.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	books: list[Book] total_pages_read: int rank: str favorite_topic: str message: str books will be modified by save_book and remove_book. These two will either change the name of the book or remove it from the list. total_pages_read will be modified by log_pages and remove_book. These two will either change the amount of pages of the book or remove it from the list. rank will be modified by log_pages and remove_book. These two will either change the rank based on the total pages read or remove it from the list. favorite_topic will be modified by set_favorite_topic. This will either change the topic.

	message will be modified by save_book and log_pages. These two change what status message is shared such as action complete!
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	title: str author: str topic: str total_pages: int pages_read: int notes: str These will not be modified directly, only removed or replaced.
Routes (minimum 7) The names of your route functions	save_book(state, title, author, topic, total_pages, notes) log_pages(state, title, pages_text) remove_book(state, title) set_favorite_topic(state, topic) index(state) add_book_page(state) book_page(state, title) rank_page(state) recommend_page(state)
Pages (minimum 5) A brief description of the pages in your site	index(state): This is the starting page. It shows the current rank, status message like an empty-shelf notice, and the different books on a loop. add_book_page(state): This is the add book page. It adds a new book to the shelf. It asks for the title, author, total pages, topic, notes, and status message something like “add a new book!”. book_page(state, title): This is a page that shows the book off. It tells the user the author, topic, notes, progress bar, pages left with messages like "Finished.” rank_page(state): This page has elements of jokes

	and places you based on how many pages you have read. It shows your current rank, total pages read, and pages to the next rank. recommend_page(state): This page says a next topic recommendation. It asks for a favorite topic based on previous read material, most read topic based on previous material, and suggests next topics with a reason for each.
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	TextBox found in the add_book page with the title, author, and total pages question. This collects information about the different books. SelectBox found in the add_book page with the topic selection. Which is then used for later functions for recommendation. TextArea found in the add_book page with the notes where they can put their ideas from the textbook. This is for people to think about what they found in the book and what is important.
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	To make sure any of the ints are ints and any strings are strings with an input validation. This will meaningfully change the logic of the site to make sure it does not crash. It will also not let you continue if there is an error. For the book can not be more than 3000 pages or something unreasonable so a cap on pages_read. This will give back an error message and change the logic to make sure the ranking page is hard to go up in. There will be rank thresholds if you have read from 0 to 100 then you are hydrogen, 100+ to 200 is helium, 200+ to 400 is Carbon, 400+ to 2000 is Oxygen, and 2000+ is gold. This changes the logic of the site because then your status changes and the position of your ranking. There will be a check for an empty shelf. This checks if the list[Books] is empty. If it is then it will return a message that says “Empty shelf.” If not then it will continue normally. This changes the logic of the site

	because nothing will work if it is empty. It checks if the total amount of pages versus the amount of pages read is the same then a message will pop up. This will meaningfully change the site by adding a message in the book_page.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	There is a loop for the index page to show off the list of books. If loops through to show off the shelf. This shows off all the books which is very important to the site because it allows you to see inside the list. There is a loop to add up the amount of pages_read to show off for rank. There is another on counting the amount of pages per topic for the recommendation.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

With no code how would i make a state dataclass for drafter for a library website	Since **Drafter** is a Python web framework and Python code is required to define a `@dataclass`, creating a state dataclass

pure without code means designing the **data structure schema** logically first—using plain English, a table, or a form builder mindset.

Here is how to structure the state for a **Library Website** state dataclass, followed by the zero-boilerplate schema you can pass to a developer or plug into your code generator.

Key Information to Track (State Schema)

For a library app in Drafter, your state needs to track four primary categories:

- **Catalog Data:** The collection of available books.
- **User Activity:** Active search queries, selected filters, or currently viewed items.
- **User Account / Checkout:** Borrowed books, wishlists, and due dates.
- **UI Navigation Status:** Status/feedback messages (e.g., "Book checked out successfully!").

Data Field Blueprint

Field Name	Data Type	Purpose / Default Value
`search_query`	Text (`str`)	Stores whatever the user typed into the search bar (Default: `""`)
`selected_category`	Text (`str`)	Current genre filter, like "All", "Fiction", "Science" (Default: `"All"`)

`books_catalog`	List of Books	Master list containing all library book records
`borrowed_books`	List of Books	Books currently checked out by the user
`wishlist`	List of Books	Books saved for later
`message`	Text (`str`)	Feedback alert shown to the user (e.g., "Item added") (Default: `""`)

Sub-Structure: The `Book` Model

To make the library state clean, a individual **Book** item should hold:

- `id`: Unique identifier (e.g., `101`)
- `title`: Book title (e.g., `"The Great Gatsby"`)
- `author`: Author name (e.g., `"F. Scott Fitzgerald"`)
- `genre`: Category (e.g., `"Classic"`)
- `is_available`: Availability flag (`True` or `False`)

what different functions can help modify a state dataclass for a libraryNO CODE

When

designing a state management system around dataclasses, the choice of update functions determines whether your library enforces strict immutability, prioritizes high performance through direct updates, or relies on structural event flows.

1. Copy-and-Replace Functions

Copy-and-replace functions

create an entirely new dataclass instance based on an existing state object, substituting only the specified fields while keeping all unchanged values intact.

- **Mechanism:** Accepts a baseline state object alongside target key-value pairs, returning a fresh, updated instance without altering the original.
- **Primary Use Case:** Enforcing pure functional state transitions and preserving past states for history, undo/redo features, or time-travel debugging.

2. Builder and Evolution Methods

Builder methods encapsulate update logic directly within the state class itself or

through dedicated builder objects.

- **Mechanism:** Exposes chained or field-specific named methods (such as `with_theme(...)` or `set_status(...)`) that validate input parameters and internally return a newly configured state instance.
- **Primary Use Case:** Object-oriented SDKs and client libraries that prioritize explicit API contracts, strict validation on field changes, and rich IDE autocompletion for library consumers.

3. Reducer Functions

Reducer functions derive the next state by combining the current state instance with an explicit action or event payload.

- **Mechanism:** Accepts `(Current State, Action/Event)` as input arguments, uses pattern matching or dispatching based on the action type, and outputs `Next State`.
- **Primary Use Case:** Event-driven architecture, state machines, and predictable application pipelines where every state change must

correspond to a logged, auditable event.

4. In-Place Mutator Functions

Mutator functions operate directly on mutable dataclass fields by updating them in memory without instantiating new objects.

- **Mechanism:** Receives a reference to the active state object and modifies its attribute values directly. Returns nothing (`void` / `None`).
- **Primary Use Case:** High-performance, real-time engines (such as game physics loops, sensory data processors, or high-frequency telemetry parsers) where allocating new memory on every state tick introduces unacceptable garbage-collection overhead.

5. Draft-Proxy (Recipe-Based) Functions

Draft-proxy functions provide the developer-friendly syntax

of direct mutation while maintaining complete underlying immutability.

- **Mechanism:** Accepts an initial state and a modifier callback function. The callback operates on a temporary "draft" proxy object, tracking changes and generating a pristine, immutable state object behind the scenes when the operation completes.
- **Primary Use Case:** Systems with deeply nested dataclasses where standard replacement syntax becomes overly verbose or difficult to read.

Architectural Comparison

Function Pattern	Immutability	Memory Cost	Predictability & Auditability	Primary Strengths
Copy-and-Replace	Strict Immutable	Moderate (Allocates new object)	High	Zero side effects; ideal for history tracking.
Builder / Evolution	Strict Immutable	Moderate (Allocates new object)	High	Type-safe, self-documenting, clean

				developer experience.
Reducer	Strict Immutable	Moderate (Allocates new object)	Very High	Centralizes business logic; enforces event-drive n patterns.
In-Place Mutator	Mutable	Very Low (Zero allocations)	Low	Maximum speed and low memory pressure for real-time systems.
Draft-Proxy	Strict Immutable	Moderate to High	High	Keeps state immutable without verbose nested update calls.

how could a loop and if statements be put into a code using python and drafter for a chemistry library

In Python chemistry applications—such as those using **RDKit**, **ASE** (Atomic Simulation Environment), or custom molecular classes (often called "drafters" or visual builders)—loops and `if` statements work together to filter, process, or construct chemical structures.

Core Concepts

- **Loops (`for` / `while`):** Iterate through atoms, bonds, residues, or a collection of candidate molecules.
- `if` **Statements:** Filter or modify objects based on chemical rules (e.g.,

atomic number, formal charge, bond order, molecular weight, or functional group presence).

Example 1: Filtering & Modifying Atoms in a Molecule (RDKit)

In this scenario, a loop inspects every atom in a molecule, and an `if` statement applies a rule (e.g., highlighting or flagging reactive sites like uncharged nitrogens).

ython

None

```
from rdkit import Chem
```

```
# Load a molecule (e.g., Caffeine)
mol = Chem.MolFromSmiles("CN1C=NC2=C1C(=O)N(C(=O)N2C)C")

reactive_nitrogen_indices = []

# Loop over every atom in the molecule
for atom in mol.GetAtoms():
    # If statement checking chemical properties
    if atom.GetSymbol() == "N" and
atom.GetFormalCharge() == 0:
        reactive_nitrogen_indices.append(atom.GetIdx())
        print(f"Atom Index {atom.GetIdx()}: Neutral
Nitrogen found")

print(f"Target atom indices for draft modification:
{reactive_nitrogen_indices}")
```

Example 2: Drafting / Building a Custom Structure Array (ASE / Custom Library)

When using a drafting tool or building atomic structures programmatically (e.g., constructing a crystal lattice or modifying a molecular framework), loops generate coordinates while `if` statements apply boundary or element-substitution conditions.

ython

None

```
# Conceptual representation of a structure drafter loop

atoms_list = []


# Loop through a 3D grid space (x, y, z) to draft a
lattice structure

for x in range(3):

    for y in range(3):

        for z in range(3):

            # If condition to substitute a central
dopant atom into the lattice

            if x == 1 and y == 1 and z == 1:

                element = "Pt"  # Platinum dopant at the
center

            else:

                element = "Au"  # Gold lattice base


            # Draft atom coordinate

            atom_data = {
```

```
                "element": element,
                "position": (x * 2.5, y * 2.5, z * 2.5)
            }
            atoms_list.append(atom_data)

print(f"Drafted {len(atoms_list)} atomic positions.")
```

Example 3: Screening a Library of SMILES Strings

When drafting a library of novel compounds, loops iterate through candidate molecules, and `if` statements filter them based on

Lipinski’s
Rule of Five
or structural
constraints.

ython

None

```
from rdkit import Chem
from rdkit.Chem import Descriptors


# Candidate SMILES strings to screen
smiles_library = [
    "CC(=O)OC1=CC=CC=C1C(=O)O",  # Aspirin
    "CN1C=NC2=C1C(=O)N(C(=O)N2C)C", # Caffeine
    "CCCCCCCCCCCCCCCCCC(=O)O"      # Stearic acid
]


passed_candidates = []
```

```
# Loop through every molecule in the draft library
for smiles in smiles_library:
    mol = Chem.MolFromSmiles(smiles)

    if mol is not None:
        mw = Descriptors.MolWt(mol)
        logp = Descriptors.MolLogP(mol)

        # If statement enforcing property criteria
        if mw < 300 and logp < 3.0:
            passed_candidates.append((smiles, mw, logp))

print("Molecules passing criteria:")
for item in passed_candidates:
    print(f"SMILES: {item[0]} | MW: {item[1]:.2f} |
LogP: {item[2]:.2f}")
```

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.